```
/* ============================================================
  ROVER 2 — ESP32-CAM
  Edge Impulse ML Object Detection + WiFi AP + TCP Alert

  Detects: HUMAN, FIRE
  Sends TCP alert to Pico W over WiFi AP

  WiFi AP: RoverCAM_AP_R2 / rover1234
  TCP Port: 5000
  Alert format: HUMAN_DETECTED  or  FIRE_DETECTED  (plain text)
  ============================================================ */

#include <Object_detection_2_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"
#include "esp_camera.h"
#include <WiFi.h>

// — Camera pins (AI Thinker) ————————————————————
#define CAMERA_MODEL_AI_THINKER
#define PWDN_GPIO_NUM    32
#define RESET_GPIO_NUM   -1
#define XCLK_GPIO_NUM     0
#define SIOD_GPIO_NUM    26
#define SIOC_GPIO_NUM    27
#define Y9_GPIO_NUM      35
#define Y8_GPIO_NUM      34
#define Y7_GPIO_NUM      39
#define Y6_GPIO_NUM      36
```

```
#define Y5_GPIO_NUM      21
#define Y4_GPIO_NUM      19
#define Y3_GPIO_NUM      18
#define Y2_GPIO_NUM       5
#define VSYNC_GPIO_NUM   25
#define HREF_GPIO_NUM    23
#define PCLK_GPIO_NUM    22

// — Frame buffer ————————————————————————————
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS  320
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS  240
#define EI_CAMERA_FRAME_BYTE_SIZE        3

// — WiFi AP config ——————————————————————————
const char* AP_SSID     = "RoverCAM_AP_R2";
const char* AP_PASSWORD  = "rover1234";
const int   TCP_PORT    = 5000;

// — Detection thresholds ————————————————————
#define DETECTION_THRESHOLD  0.6f
#define COOLDOWN_MS        5000UL

// — Globals ——————————————————————————————————
static bool   debug_nn      = false;
static bool   is_initialised = false;
uint8_t      *snapshot_buf;

WiFiServer  tcpServer(TCP_PORT);
```

```
WiFiClient tcpClient;
uint32_t   lastHumanAlertMs = 0;
uint32_t   lastFireAlertMs  = 0;

// ── Camera config ──────────────────────────────────────────
static camera_config_t camera_config = {
  .pin_pwdn     = PWDN_GPIO_NUM,
  .pin_reset    = RESET_GPIO_NUM,
  .pin_xclk     = XCLK_GPIO_NUM,
  .pin_sscb_sda = SIOD_GPIO_NUM,
  .pin_sscb_scl = SIOC_GPIO_NUM,
  .pin_d7       = Y9_GPIO_NUM,
  .pin_d6       = Y8_GPIO_NUM,
  .pin_d5       = Y7_GPIO_NUM,
  .pin_d4       = Y6_GPIO_NUM,
  .pin_d3       = Y5_GPIO_NUM,
  .pin_d2       = Y4_GPIO_NUM,
  .pin_d1       = Y3_GPIO_NUM,
  .pin_d0       = Y2_GPIO_NUM,
  .pin_vsync    = VSYNC_GPIO_NUM,
  .pin_href     = HREF_GPIO_NUM,
  .pin_pclk     = PCLK_GPIO_NUM,
  .xclk_freq_hz = 20000000,
  .ledc_timer   = LEDC_TIMER_0,
  .ledc_channel = LEDC_CHANNEL_0,
  .pixel_format = PIXFORMAT_JPEG,
  .frame_size   = FRAMESIZE_QVGA,
  .jpeg_quality = 12,
```

```
  .fb_count     = 1,
  .fb_location  = CAMERA_FB_IN_PSRAM,
  .grab_mode    = CAMERA_GRAB_WHEN_EMPTY,
};

// — Forward declarations ————————————————————————
bool ei_camera_init(void);
void ei_camera_deinit(void);
bool ei_camera_capture(uint32_t w, uint32_t h, uint8_t *buf);
void handleTcpClient();
void sendDetectionAlert(const char* label);

// — Accept Pico W TCP connection ————————————————
void handleTcpClient() {
  if (!tcpClient || !tcpClient.connected()) {
    tcpClient = tcpServer.accept();
    if (tcpClient) {
      Serial.println("[CAM-R2] Pico W connected!");
      tcpClient.println("CONNECTED_TO_ROVER2_CAM");
    }
  }
}

// — Send plain-text alert to Pico W ——————————————
void sendDetectionAlert(const char* label) {
  uint32_t now = millis();
  String  msg = "";
```

```
  if (strcasecmp(label, "human") == 0) {
    if (now - lastHumanAlertMs < COOLDOWN_MS) return;
    lastHumanAlertMs = now;
    msg = "HUMAN_DETECTED";

  } else if (strcasecmp(label, "fire") == 0) {
    if (now - lastFireAlertMs < COOLDOWN_MS) return;
    lastFireAlertMs = now;
    msg = "FIRE_DETECTED";

  } else {
    return;
  }

  if (tcpClient && tcpClient.connected()) {
    tcpClient.println(msg);
    Serial.print("[CAM-R2] Sent → "); Serial.println(msg);
  } else {
    Serial.println("[CAM-R2] Pico W not connected — alert dropped");
  }
}

// — Setup ————————————————————————————————————
void setup() {
  Serial.begin(115200);

  WiFi.mode(WIFI_AP);
  WiFi.softAP(AP_SSID, AP_PASSWORD);
```

```
  Serial.print("[CAM-R2] AP: "); Serial.print(AP_SSID);
  Serial.print("  IP: "); Serial.println(WiFi.softAPIP());

  tcpServer.begin();
  Serial.print("[CAM-R2] TCP server port "); Serial.println(TCP_PORT);

  while (!Serial);
  if (!ei_camera_init()) {
    ei_printf("ERR: Camera init failed\r\n");
  } else {
    ei_printf("Camera OK\r\n");
  }
  ei_printf("Starting inference in 2s...\n");
  ei_sleep(2000);
}

// — Main loop ————————————————————————————————————
void loop() {
  handleTcpClient();

  if (ei_sleep(5) != EI_IMPULSE_OK) return;

  snapshot_buf = (uint8_t*)malloc(
    EI_CAMERA_RAW_FRAME_BUFFER_COLS *
    EI_CAMERA_RAW_FRAME_BUFFER_ROWS *
    EI_CAMERA_FRAME_BYTE_SIZE);
  if (!snapshot_buf) { ei_printf("ERR: malloc failed\n"); return; }
```

```
  ei::signal_t signal;
  signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
  signal.get_data     = &ei_camera_get_data;

  if (!ei_camera_capture(EI_CLASSIFIER_INPUT_WIDTH,
                         EI_CLASSIFIER_INPUT_HEIGHT, snapshot_buf)) {
    ei_printf("ERR: Capture failed\r\n");
    free(snapshot_buf); return;
  }

  ei_impulse_result_t result = {0};
  EI_IMPULSE_ERROR err = run_classifier(&signal, &result, debug_nn);
  if (err != EI_IMPULSE_OK) {
    ei_printf("ERR: Classifier failed (%d)\n", err);
    free(snapshot_buf); return;
  }

  ei_printf("Predictions (DSP:%dms Class:%dms Anom:%dms):\n",
            result.timing.dsp, result.timing.classification, result.timing.anomaly);

#if EI_CLASSIFIER_OBJECT_DETECTION == 1
  for (uint32_t i = 0; i < result.bounding_boxes_count; i++) {
    ei_impulse_result_bounding_box_t bb = result.bounding_boxes[i];
    if (bb.value == 0) continue;
    ei_printf("  %s (%.2f) [x:%u y:%u w:%u h:%u]\r\n",
              bb.label, bb.value, bb.x, bb.y, bb.width, bb.height);
    if (bb.value >= DETECTION_THRESHOLD) sendDetectionAlert(bb.label);
  }
```

```
#else
  for (uint16_t i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    ei_printf("  %s: %.5f\r\n",
          ei_classifier_inferencing_categories[i],
          result.classification[i].value);
    if (result.classification[i].value >= DETECTION_THRESHOLD)
      sendDetectionAlert(ei_classifier_inferencing_categories[i]);
  }
#endif

#if EI_CLASSIFIER_HAS_ANOMALY
  ei_printf("Anomaly: %.3f\r\n", result.anomaly);
#endif

  free(snapshot_buf);
}

// — Camera functions ————————————————————————————
bool ei_camera_init(void) {
  if (is_initialised) return true;
  esp_err_t err = esp_camera_init(&camera_config);
  if (err != ESP_OK) {
    Serial.printf("Camera init error 0x%x\n", err);
    return false;
  }
  sensor_t *s = esp_camera_sensor_get();
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1);
```

```
    s->set_brightness(s, 1);
    s->set_saturation(s, 0);
  }
  is_initialised = true;
  return true;
}

void ei_camera_deinit(void) {
  esp_camera_deinit();
  is_initialised = false;
}

bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf) {
  if (!is_initialised) { ei_printf("ERR: Camera not init\r\n"); return false; }
  camera_fb_t *fb = esp_camera_fb_get();
  if (!fb) { ei_printf("ERR: Capture failed\n"); return false; }
  bool ok = fmt2rgb888(fb->buf, fb->len, PIXFORMAT_JPEG, snapshot_buf);
  esp_camera_fb_return(fb);
  if (!ok) { ei_printf("ERR: Conversion failed\n"); return false; }
  if (img_width != EI_CAMERA_RAW_FRAME_BUFFER_COLS ||
    img_height != EI_CAMERA_RAW_FRAME_BUFFER_ROWS) {
    ei::image::processing::crop_and_interpolate_rgb888(
      out_buf, EI_CAMERA_RAW_FRAME_BUFFER_COLS,
      EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
      out_buf, img_width, img_height);
  }
  return true;
}
```

```
static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr) {
  size_t pixel_ix = offset * 3;
  for (size_t i = 0; i < length; i++) {
    out_ptr[i] = (snapshot_buf[pixel_ix+2] << 16) |
           (snapshot_buf[pixel_ix+1] << 8)  |
            snapshot_buf[pixel_ix];
    pixel_ix += 3;
  }
  return 0;
}

#if !defined(EI_CLASSIFIER_SENSOR) || EI_CLASSIFIER_SENSOR !=
EI_CLASSIFIER_SENSOR_CAMERA
#error "Invalid model for current sensor"
#endif
```